SQL — Sélection de données

Sommaire

1. Structure générale SELECT 2
2. DISTINCT 4
3. WHERE 5
4. Opérateurs de comparaison 6
5. AND & OR 7
6. BETWEEN ... AND 9
7. LIKE 11
8. Fonctions d'agrégation 13
9. GROUP BY 14
10. HAVING 15
11. ORDER BY 16
12. Jointures SQL 18
13. JOIN 19
14. LEFT JOIN 20
15. RIGHT JOIN 21
16. Notation table.colonne 22
17. Joindre trois tables 23
18. Commentaires en SQL 24

1. Structure générale SELECT

La commande SQL `SELECT` est commune car il est très fréquent de devoir lire les données issues d'une base de données :

- Filtrer pour ne sélectionner que certains enregistrements
- Classer (ordonner) les résultats
- Grouper les résultats pour faire uniquement des statistiques (note moyenne, prix le plus élevé)
- Joindre une autre table aux résultats

Voici l'ordre des éléments d'une requête de sélection de données (quand toutes les parties sont présentes) :

```
SELECT …
FROM table
JOIN une_autre_table ON condition
WHERE condition
GROUP BY expression
HAVING condition
ORDER BY expression
```

1.1. Commande basique

L'utilisation basique de cette commande s'effectue de la manière suivante :

```
SELECT colonne
FROM table
```

Cette requête va sélectionner (`SELECT`) la colonne provenant (`FROM`) de `table` .

1.2. Exemple

Imaginons une base de données qui contient des informations sur les clients d'une entreprise.

Table « clients_villes » :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	Pierre	Leroy	Marseille
5	Marie	Leroy	Nice

Si l'on veut avoir la liste de toutes les villes des clients, il suffit d'effectuer la requête suivante :

```
SELECT ville
FROM clients_villes
```

Résultat :

ville
Paris
Nantes
Lyon
Marseille
Nice

1.3. Obtenir plusieurs colonnes

Avec la même table il est possible de lire plusieurs colonnes à la fois. Il suffit de séparer les noms des colonnes par des virgules.

```
SELECT prenom, nom
FROM clients_villes
```

Résultat :

prenom	nom
Pierre	Dupond
Sabrina	Durand
Julien	Martin
Pierre	Leroy
Marie	Leroy

1.4. Obtenir toutes les colonnes d'une table

Il est possible de retourner automatiquement toutes les colonnes d'une table sans avoir à connaître le nom de toutes les colonnes en utilisant le caractère `*` :

```
SELECT * FROM clients_villes
```

Cette requête retourne exactement les mêmes colonnes qu'il y a dans la base de données. C'est une bonne idée de commencer à écrire une requête en utilisant `*`, pour ensuite raffiner la requête en extrayant uniquement les colonnes souhaitées.

2. DISTINCT

L'utilisation de la commande `SELECT` en SQL permet de lire toutes les données d'une ou plusieurs colonnes. Cette commande peut potentiellement afficher des doublons si la même donnée est présente plusieurs fois. Pour éviter les doublons, on utilise `DISTINCT` .

2.1. Syntaxe

```
SELECT DISTINCT colonne
FROM table
```

Cette requête sélectionne le champ `colonne` de `table` en évitant de retourner des doublons.

2.2. Exemple

Prenons le cas concret d'une table « clients_villes » :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	Pierre	Leroy	Paris
5	Marie	Leroy	Nantes

Sans `DISTINCT` , Pierre apparaît deux fois :

```
SELECT prenom
FROM clients_villes
```

Résultat :

prenom
Pierre
Sabrina
Julien
Pierre
Marie

Avec `DISTINCT` , chaque valeur n'apparaît qu'une seule fois :

```
SELECT DISTINCT prenom
FROM clients_villes
```

Résultat :

prenom
Pierre
Sabrina
Julien
Marie

3. WHERE

La commande `WHERE` dans une requête SQL permet d'extraire les lignes d'une base de données qui respectent une condition.

3.1. Syntaxe

```sql
SELECT nom_colonnes
FROM nom_table
WHERE condition
```

3.2. Exemple

Imaginons une base de données appelée « clients_commandes » qui contient le nom des clients, le nombre de commandes qu'ils ont effectuées et leur ville.

id	nom	nbr_commande	ville
1	Paul	3	Paris
2	Maurice	0	Rennes
3	Joséphine	1	Toulouse
4	Gérard	7	Paris

Pour obtenir seulement la liste des clients qui habitent à Paris :

```sql
SELECT *
FROM clients_commandes
WHERE ville = 'Paris'
```

Résultat :

id	nom	nbr_commande	ville
1	Paul	3	Paris
4	Gérard	7	Paris

Attention : les comparaisons sont sensibles aux majuscules/minuscules. Si on avait par exemple écrit `WHERE ville = 'paris'`, aucun résultat n'aurait été retourné.

4. Opérateurs de comparaison

Il existe plusieurs opérateurs de comparaison. La liste ci-jointe présente quelques-uns des opérateurs les plus couramment utilisés :

Opérateur	Description
`=`	Égal
`<>`	Pas égal
`!=`	Pas égal
`>`	Supérieur à
`<`	Inférieur à
`>=`	Supérieur ou égal à
`<=`	Inférieur ou égal à
`IN`	Liste de plusieurs valeurs possibles
`BETWEEN … AND …`	Valeur comprise dans un intervalle donné (utile pour les nombres ou dates)
`LIKE`	Recherche en spécifiant le début, milieu ou fin d'un mot
`IS NULL`	Valeur est nulle
`IS NOT NULL`	Valeur n'est pas nulle

On peut comparer une colonne avec une valeur (par exemple `age >= 18` ou `ville = 'genève'`).

On peut aussi comparer deux colonnes entre elles (par exemple `recettes > dépenses`).

Comparer deux valeurs entre elles n'est pas utile (par exemple la condition `'A' = 'B'` n'est jamais satisfaite, et la requête retournera toujours un résultat vide).

5. AND & OR

Une requête SQL peut être restreinte à l'aide de la condition `WHERE` . Les opérateurs logiques `AND` et `OR` peuvent être utilisés pour combiner plusieurs conditions.

5.1. Syntaxe d'utilisation des opérateurs AND et OR

L'opérateur `AND` permet de s'assurer que la condition1 ET la condition2 sont vraies :

```
SELECT colonnes FROM table
WHERE condition1 AND condition2
```

L'opérateur `OR` vérifie quant à lui que la condition1 OU la condition2 sont vraies :

```
SELECT colonnes FROM table
WHERE condition1 OR condition2
```

Ces opérateurs peuvent être combinés à l'infini et mélangés :

```
SELECT nom_colonnes FROM nom_table
WHERE condition1 AND (condition2 OR condition3)
```

Attention : il faut penser à utiliser des parenthèses lorsque c'est nécessaire. Cela permet d'éviter les erreurs et facilite la lecture de la requête.

5.2. Exemple de données

Pour illustrer les prochaines commandes, nous allons considérer la table « produits » suivante :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35
3	souris	informatique	16	30
4	crayon	fourniture	147	2

5.3. Opérateur AND

```
SELECT * FROM produits
WHERE categorie = 'informatique' AND stock < 20
```

Résultat :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
3	souris	informatique	16	30

5.4. Opérateur OR

Pour filtrer les données pour avoir uniquement les données sur les produits « ordinateur » ou « clavier » :

```sql
SELECT * FROM produits
WHERE nom = 'ordinateur' OR nom = 'clavier'
```

Résultat :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
2	clavier	informatique	32	35

5.5. Combiner AND et OR

Les opérateurs peuvent être combinés pour effectuer de puissantes recherches :

```sql
SELECT * FROM produits
WHERE (categorie = 'informatique' AND stock < 20)
   OR (categorie = 'fourniture' AND stock < 200)
```

Résultat :

id	nom	categorie	stock	prix
1	ordinateur	informatique	5	950
3	souris	informatique	16	30
4	crayon	fourniture	147	2

Noter l'utilisation de parenthèses pour contrôler l'ordre d'application des opérateurs `AND` et `OR`.

6. BETWEEN ... AND ...

L'opérateur `BETWEEN` est utilisé dans une requête SQL pour sélectionner un intervalle de données dans une requête utilisant `WHERE` .

6.1. Syntaxe

```
SELECT * FROM table
WHERE colonne BETWEEN 'valeur1' AND 'valeur2'
```

La requête retournera toutes les lignes dont la valeur de la colonne est comprise entre valeur1 et valeur2, **bornes incluses**.

6.2. Exemple : filtrer entre 2 dates

Imaginons une table « utilisateurs » qui contient les membres d'une application en ligne.

id	nom	prenom	date_inscription	tarif_total
1	Durand	Maurice	2012-02-05	145
2	Dupond	Fabrice	2012-02-07	65
3	Durand	Fabienne	2012-02-13	90
4	Dubois	Chloé	2012-02-16	98
5	Dubois	Simon	2012-02-23	27

Pour obtenir les membres qui se sont inscrits entre le 10 et le 16 février 2012 :

```
SELECT * FROM utilisateurs
WHERE date_inscription BETWEEN '2012-02-10' AND '2012-02-16'
```

Résultat :

id	nom	prenom	date_inscription	tarif_total
3	Durand	Fabienne	2012-02-13	90
4	Dubois	Chloé	2012-02-16	98

6.3. Exclure avec NOT BETWEEN

Pour obtenir tous les résultats dont l'identifiant n'est pas situé entre 2 et 4 :

```
SELECT * FROM utilisateurs
WHERE id NOT BETWEEN 2 AND 4
```

Résultat :

id	nom	prenom	date_inscription	tarif_total
1	Durand	Maurice	2012-02-05	145
5	Dubois	Simon	2012-02-23	27

6.4. Bornes incluses dans l'intervalle

L'opérateur `BETWEEN` inclut les bornes. Cette requête :

```sql
SELECT * FROM table
WHERE colonne BETWEEN 'valeur1' AND 'valeur2'
```

Est équivalente à :

```sql
SELECT * FROM table
WHERE colonne >= 'valeur1' AND colonne <= 'valeur2'
```

7. LIKE

L'opérateur `LIKE` est utilisé dans la clause `WHERE` des requêtes SQL. Ce mot-clé permet d'effectuer une recherche sur un modèle particulier.

7.1. Syntaxe

```
SELECT * FROM table
WHERE colonne LIKE modele
```

Le « modèle » ressemble très généralement à l'un des exemples suivants :

- `LIKE '%er'` : le caractère `%` est un caractère joker qui remplace tous les autres caractères. Ce modèle permet de rechercher tous les enregistrements qui se terminent par « er ».
- `LIKE 'M%'` : ce modèle permet de rechercher tous les enregistrements qui commencent par un « M ».
- `LIKE '%et%'` : ce modèle est utilisé pour rechercher tous les enregistrements qui contiennent la séquence « et ».
- `LIKE 'pa%on'` : ce modèle permet de rechercher les chaînes qui commencent par « pa » et qui se terminent par « on », comme « pantalon » ou « pardon ».

7.2. Exemple

Imaginons une table « clients_villes » :

identifiant	prenom	nom	ville
1	Pierre	Dupond	Paris
2	Sabrina	Durand	Nantes
3	Julien	Martin	Lyon
4	Pierre	Leroy	Marseille
5	Marie	Leroy	Nice

Obtenir les résultats qui commencent par « N »

```
SELECT * FROM clients_villes
WHERE ville LIKE 'N%'
```

Avec cette requête, seuls les enregistrements suivants seront retournés :

id	prenom	nom	ville
2	Sabrina	Durand	Nantes
5	Marie	Leroy	Nice

Obtenir les résultats qui se terminent par « e »

```
SELECT * FROM clients_villes
WHERE ville LIKE '%e'
```

Résultat :

id	prenom	nom	ville
4	Pierre	Leroy	Marseille
5	Marie	Leroy	Nice

8. Fonctions d'agrégation

Il existe plusieurs fonctions qui peuvent être utilisées pour calculer une valeur concernant plusieurs enregistrements. Ces fonctions s'appliquent à une colonne et retournent une valeur unique.

8.1. Exemple d'utilisation

Prenons en considération une table « achats » qui résume les ventes d'une boutique :

id	client	prix	date
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Pour obtenir le montant total pour Pierre :

```
SELECT client, SUM(prix)
FROM achat
WHERE client = 'Pierre'
```

On obtient :

Client	SUM(prix)
Pierre	262

8.2. Liste des fonctions d'agrégation

Les principales fonctions d'agrégation sont :

- `COUNT(…)` pour compter le nombre de lignes concernées. Permet par exemple de savoir combien d'achats ont été effectués par un client.
- `SUM(…)` pour calculer la somme de plusieurs valeurs. Permet par exemple de connaître le total de tous les achats d'un client.
- `MIN(…)` pour récupérer la plus petite valeur. Utile par exemple pour connaître la date du premier achat d'un client.
- `MAX(…)` pour récupérer la plus grande valeur. Pratique pour connaître l'achat le plus cher.
- `AVG(…)` pour calculer la moyenne d'un ensemble de valeurs. Permet de connaître le prix du panier moyen pour chaque client.

9. GROUP BY

La commande `GROUP BY` est utilisée en SQL pour grouper plusieurs résultats et utiliser une fonction d'agrégation sur un groupe de résultats.

9.1. Syntaxe d'utilisation de GROUP BY

```
SELECT colonne1, fonction_d_agregation(colonne2)
FROM table
GROUP BY colonne1
```

9.2. Exemple d'utilisation

Prenons en considération une table « achats » :

id	client	prix	date
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Pour obtenir le montant total pour chaque client :

```
SELECT client, SUM(prix) FROM achat
GROUP BY client
```

La fonction `SUM()` additionne la valeur de chaque prix pour un même client. Le résultat est :

client	SUM(prix)
Pierre	262
Simon	47
Marie	38

Attention — Quand on groupe il ne faut plus accéder aux attributs de chaque rangée. Par exemple la requête suivante n'a pas de sens car `prix` va simplement retourner le prix d'*un* achat arbitraire du client :

```
SELECT client, prix FROM achat
GROUP BY client
```

10. HAVING

La clause `HAVING` en SQL permet de filtrer des groupements. Elle ressemble à la clause `WHERE` (qui filtre des lignes), sauf que `HAVING` filtre des groupes.

10.1. Syntaxe

```
SELECT colonne1, SUM(colonne2) FROM table
GROUP BY colonne1
HAVING fonction(colonne2) operateur valeur
```

Cela permet de n'inclure que les groupes pour lesquels la condition `HAVING` est respectée.

10.2. Exemple

Imaginons une table « achats » qui contient les achats de différents clients avec le coût du panier pour chaque achat.

id	client	prix	date_achat
1	Pierre	102	2012-10-23
2	Simon	47	2012-10-27
3	Marie	18	2012-11-05
4	Marie	20	2012-11-14
5	Pierre	160	2012-12-03

Pour récupérer la liste des clients qui ont commandé plus de 40€ au total :

```
SELECT client, SUM(prix) FROM achat
GROUP BY client
HAVING SUM(prix) > 40
```

Résultat :

client	SUM(prix)
Pierre	262
Simon	47

La cliente « Marie » a cumulé 38€ d'achat (un achat de 18€ et un autre de 20€) ce qui est inférieur à la limite de 40€ imposée par le `HAVING`.

11. ORDER BY

La clause `ORDER BY` permet d'ordonner les lignes dans un résultat d'une requête SQL. Il est possible d'ordonner par ordre ascendant (par défaut) ou DESCendant. Il est également possible d'ordonner d'après plusieurs colonnes.

11.1. Syntaxe

Une requête avec tri sur une seule colonne :

```
SELECT colonne1, colonne2 FROM table
ORDER BY colonne1
```

Par défaut les résultats sont classés par ordre ascendant. Il est possible d'inverser l'ordre avec `DESC` , et de trier sur plusieurs colonnes :

```
SELECT colonne1, colonne2, colonne3 FROM table
ORDER BY colonne1 DESC, colonne2
```

11.2. Exemples

Nous allons considérer une base avec une table « utilisateurs » :

id	nom	prenom	date_inscription	tarif_total
1	Durand	Maurice	2012-02-05	145
2	Dupond	Fabrice	2012-02-07	65
3	Durand	Fabienne	2012-02-13	90
4	Dubois	Chloé	2012-02-16	98
5	Dubois	Simon	2012-02-23	27

Pour récupérer la liste par ordre alphabétique du nom de famille :

```
SELECT *
FROM utilisateurs
ORDER BY nom
```

Résultat :

id	nom	prenom	date_inscription	tarif_total
4	Dubois	Chloé	2012-02-16	98
5	Dubois	Simon	2012-02-23	27
2	Dupond	Fabrice	2012-02-07	65
1	Durand	Maurice	2012-02-05	145
3	Durand	Fabienne	2012-02-13	90

En spécifiant plusieurs colonnes de tri, les utilisateurs sont triés par nom de famille, puis pour les mêmes noms par date d'inscription décroissante :

```sql
SELECT *
FROM utilisateurs
ORDER BY nom, date_inscription DESC
```

Résultat :

id	nom	prenom	date_inscription	tarif_total
5	Dubois	Simon	2012-02-23	27
4	Dubois	Chloé	2012-02-16	98
2	Dupond	Fabrice	2012-02-07	65
3	Durand	Fabienne	2012-02-13	90
1	Durand	Maurice	2012-02-05	145

On peut aussi ordonner le résultat d'un groupement. Par exemple pour afficher les noms de famille les plus courants en premier :

```sql
SELECT nom, count(*)
FROM utilisateurs
GROUP BY nom
ORDER BY count(*) DESC
```

Résultat :

nom	count(*)
Durand	2
Dubois	2
Dupond	1

C'est la raison pour laquelle `ORDER BY` vient après des éventuels `GROUP BY` et `HAVING` dans une requête SQL.

12. Jointures SQL

Les jointures en SQL permettent de combiner plusieurs tables dans une même requête. Cela permet d'exploiter la puissance des bases de données relationnelles.

Les jointures consistent à associer des lignes de 2 tables en associant l'égalité des valeurs d'une colonne d'une première table avec la valeur d'une colonne d'une seconde table.

On peut imaginer qu'un site web possède une table pour les articles et une autre table pour les commentaires associés à chaque article. Une jointure permettrait d'afficher les articles avec leurs commentaires.

12.1. Modèle de données

Imaginons une application qui contient une table utilisateur ainsi qu'une table commande qui contient les commandes effectuées par les utilisateurs.

Table utilisateur :

id_utilisateur	prenom	nom	email
1	Aimée	Marechal	amarechal@x.co
2	Esmée	Lefort	elefort@x.co
3	Marine	Prevost	mprevost@x.co
4	Luc	Rolland	lrolland@x.co

Table commande :

id_commande	l_utilisateur	date_achat	montant
1	1	2013-01-23	203.14
2	1	2013-02-14	124.00
3	2	2013-02-17	149.45
4	3	2013-02-21	235.35
5	6	2013-03-02	47.58

13. JOIN

Dans le langage SQL la commande `JOIN` est un type de jointure très commune pour lier plusieurs tables entre elles. Cette commande retourne les enregistrements lorsqu'il y a au moins une correspondance dans les deux tables.

13.1. Syntaxe

```
SELECT colonnes
FROM table1
JOIN table2 ON table1.id = table2.fk_id
```

La syntaxe sélectionne les enregistrements des tables table1 et table2 lorsque la colonne `id` de table1 est égale à la colonne `fk_id` de table2.

13.2. Exemple

On a les données décrites dans la section Jointures SQL. Pour afficher toutes les commandes associées aux utilisateurs :

```
SELECT *
FROM utilisateur
JOIN commande ON id_utilisateur = l_utilisateur
```

Résultat :

id_utilisateur	prenom	nom	email	id_commande	l_utilisateur	date_achat	montant
1	Aimée	Marechal	amarechal@x.co	1	1	2013-01-23	203.14
1	Aimée	Marechal	amarechal@x.co	2	1	2013-02-14	124.00
2	Esmée	Lefort	elefort@x.co	3	2	2013-02-17	149.45
3	Marine	Prevost	mprevost@x.co	4	3	2013-02-21	235.35

À gauche toutes les colonnes de la table utilisateur, à droite toutes les colonnes de la table commande. Les tables ont été jointes sur les champs `id_utilisateur` et `l_utilisateur`.

Noter que :

- L'utilisateur 1 est affiché deux fois parce qu'il a fait 2 commandes.
- L'utilisateur 4 n'est pas affiché puisqu'il n'y a pas de commande associée à cet utilisateur.

14. LEFT JOIN

Dans le langage SQL, la commande `LEFT JOIN` est un type de jointure entre 2 tables.

Cela permet de lister tous les résultats de la table de gauche (table1), même s'il n'y a pas de correspondance dans la table de droite (table2).

14.1. Syntaxe

```
SELECT colonnes FROM table1
LEFT JOIN table2 ON table1.id = table2.fk_id
```

14.2. Exemple

On a les données décrites dans la section Jointures SQL. Pour lister tous les utilisateurs avec leurs commandes et afficher également les utilisateurs qui n'ont pas effectué de commande :

```
SELECT id_utilisateur, prenom, nom, id_commande, date_achat, montant
FROM utilisateur
LEFT JOIN commande ON id_utilisateur = l_utilisateur
```

Résultat :

id_utilisateur	prenom	nom	id_commande	date_achat	montant
1	Aimée	Marechal	1	2013-01-23	203.14
1	Aimée	Marechal	2	2013-02-14	124.00
2	Esmée	Lefort	3	2013-02-17	149.45
3	Marine	Prevost	4	2013-02-21	235.35
4	Luc	Rolland	NULL	NULL	NULL

La dernière ligne montre un utilisateur qui n'a effectué aucune commande. Les informations de la commande sont à `NULL`.

15. RIGHT JOIN

En SQL, la commande `RIGHT JOIN` est un type de jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de droite (table2), même s'il n'y a pas de correspondance avec la table de gauche (table1).

15.1. Syntaxe

```
SELECT colonnes FROM table1
RIGHT JOIN table2 ON table1.id = table2.fk_id
```

15.2. Exemple

On a les données décrites dans la section Jointures SQL. Pour afficher toutes les commandes avec le nom de l'utilisateur correspondant :

```
SELECT id_utilisateur, prenom, nom, id_commande, l_utilisateur, date_achat,
montant
FROM utilisateur
RIGHT JOIN commande ON id_utilisateur = l_utilisateur
```

Résultat :

id_utilisateur	prenom	nom	id_commande	l_utilisateur	date_achat	montant
1	Aimée	Marechal	1	1	2013-01-23	203.14
1	Aimée	Marechal	2	1	2013-02-14	124.00
2	Esmée	Lefort	3	2	2013-02-17	149.45
3	Marine	Prevost	4	3	2013-02-21	235.35
NULL	NULL	NULL	5	6	2013-03-02	47.58

La commande 5 est liée à l'utilisateur 6, or cet utilisateur n'existe pas ou n'existe plus. Grâce à `RIGHT JOIN`, la commande est tout de même affichée mais les informations liées à l'utilisateur sont remplacées par `NULL`.

16. Notation table.colonne

La notation `table.colonne` permet de préciser de quelle table provient une colonne dans une requête avec plusieurs tables.

16.1. Quand c'est optionnel

Quand les noms de colonnes sont uniques d'une table à l'autre, la notation est optionnelle. Ces deux requêtes sont équivalentes :

```
SELECT id_utilisateur, prenom, nom, id_commande, date_achat
FROM utilisateur
JOIN commande ON id_utilisateur = l_utilisateur
```

```
SELECT utilisateur.id_utilisateur, utilisateur.prenom, utilisateur.nom,
       commande.id_commande, commande.date_achat
FROM utilisateur
JOIN commande ON utilisateur.id_utilisateur = commande.l_utilisateur
```

16.2. Quand c'est nécessaire

Quand deux tables ont une colonne du même nom, SQL ne sait pas laquelle utiliser et retourne une erreur. Imaginons une table `artiste` et une table `album`, toutes deux avec une colonne `id` :

Table artiste :

id	nom
1	Daft Punk
2	Björk

Table album :

id	titre	l_artiste
1	Discovery	1
2	Homework	1
3	Vespertine	2

```
-- ERREUR : "id" est ambiguë (dans SELECT et dans ON)
SELECT id, nom, titre
FROM artiste
JOIN album ON id = l_artiste
```

Il faut préciser de quelle table provient `id` :

```
SELECT artiste.id, nom, titre
FROM artiste
JOIN album ON artiste.id = l_artiste
```

Résultat :

id	nom	titre
1	Daft Punk	Discovery
1	Daft Punk	Homework
2	Björk	Vespertine

17. Joindre trois tables

Pour joindre trois tables, on ajoute simplement un deuxième `JOIN` après le premier.

Reprenons les tables `utilisateur` et `commande` de la section Jointures SQL, et ajoutons une table `programme_de_fidelite` qui associe un rabais à certains utilisateurs :

Table programme_de_fidelite :

id_fidelite	l_utilisateur	rabais_pourcent
1	1	10
2	2	5
3	3	15

Pour afficher les commandes avec le rabais de chaque utilisateur :

```
SELECT id_utilisateur, prenom, nom,
       id_commande, montant,
       rabais_pourcent
FROM utilisateur
JOIN commande ON id_utilisateur = commande.l_utilisateur
JOIN programme_de_fidelite ON id_utilisateur =
programme_de_fidelite.l_utilisateur
```

`l_utilisateur` existe dans les deux tables jointes, donc le préfixe de table est nécessaire dans les deux clauses `ON` (cf. section Notation table.colonne).

Résultat :

id_utilisateur	prenom	nom	id_commande	montant	rabais_pourcent
1	Aimée	Marechal	1	203.14	10
1	Aimée	Marechal	2	124.00	10
2	Esmée	Lefort	3	149.45	5
3	Marine	Prevost	4	235.35	15

18. Commentaires en SQL

Il peut être intéressant d'insérer des commentaires dans les requêtes SQL pour mieux s'y retrouver lorsqu'il y a de grosses requêtes.

18.1. Commentaire double tiret : –

Le double tiret permet de faire un commentaire jusqu'à la fin de la ligne.

```
SELECT *         -- tout sélectionner
FROM commandes   -- dans la table "commandes"

-- On peut aussi évidemment les écrire seuls sur une ligne
```

18.2. Commentaire multi-ligne : /* et */

Le commentaire multi-ligne permet d'écrire des commentaires plus longs.

```
/* Vous trouverez ci-dessous
   une véritable requête de professionnel
*/
SELECT *
FROM commandes
```